LLM EVALUATION

DATASETS

METRICS

EVALUATION

ANALYSIS

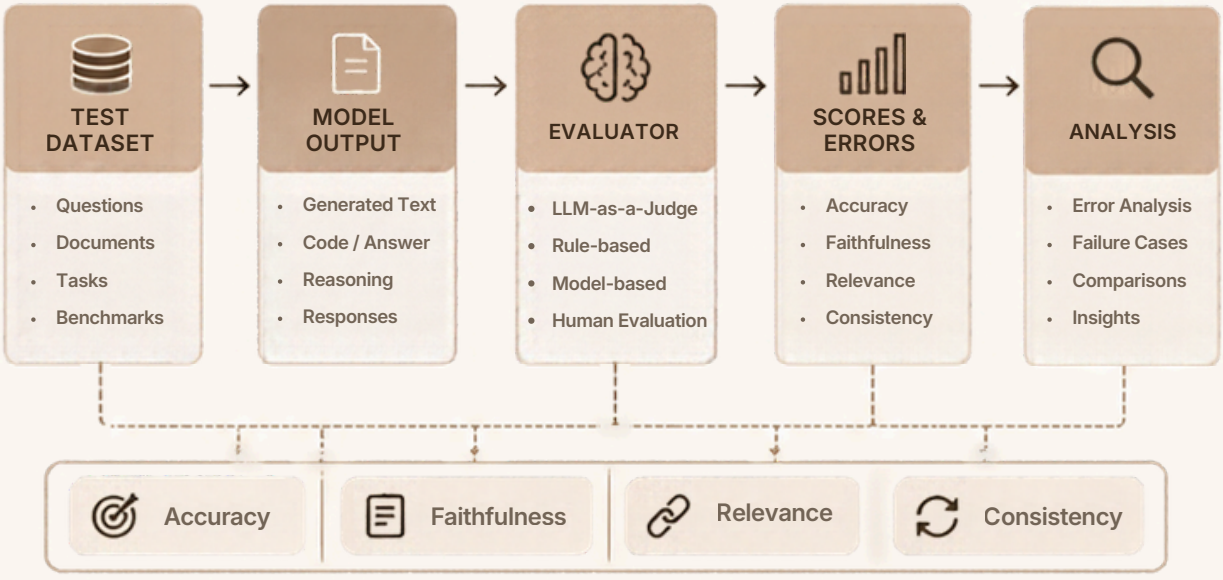
EXPERIMENTS

LLM EVALUATION · CHAPTER 04

WHY MULTIPLE EVALS

Failure Points, Eval Levels, and Risk Categories

Why one eval pipeline is never enough: component, workflow, and application-level evals across quality, safety, and operations.



WHAT THIS SERIES COVERS

- Evaluation Design
- Metrics & Rubrics
- LLM-as-a-Judge
- Regression & Error Analysis

MEASURE
UNDERSTAND
IMPROVE

PREMIUM REVISION NOTES

CONCEPTS · IMPLEMENTATION · DIAGRAMS · INTERVIEW PREP

Why Multiple Evals

RECAP — CHAPTER 3

Chapter 3 walked through the nine-step application evaluation workflow, from defining the task and success criteria, through a golden dataset and an evaluation method, to running, analyzing, improving, deploying and monitoring. Its last point was that one LLM application usually carries *several* evals rather than one. This chapter explains why.

1. Why One Eval Pipeline Is Not Enough

The statement that opens this lecture is the closing line of the previous one: **a single LLM-based application generally has several evals**. Building an LLM application does not mean attaching one evaluation pipeline to it; it means attaching several. The lecture gives two independent reasons for this.

Reason	What it means
Multiple failure points	An application is built from several parts and workflows, and each can fail independently or in combination.
Multiple risk categories	Even for one part, “good” has several dimensions: quality, safety and operations.

The lecture proves the first reason with a worked RAG example (Sections 2–4) and then introduces the second (Sections 5–6). Sections 2–4 are therefore about *where* an application can fail, and Sections 5–6 are about *what kind* of failure to look for.

IMPORTANT

“Several evals” does not mean the same check repeated. Each eval pipeline targets a different failure point or a different risk category, so each answers a different question about the same application.

↻ Quick Revision — Why Multiple Evals

- A single LLM application normally has several eval pipelines, not one.
- Reason 1: an application has multiple failure points (components, workflows, the whole app).
- Reason 2: each failure point has multiple risk categories (quality, safety, operations).
- Each eval pipeline answers a different question about the same application.

2. Failure Points in a RAG Application

The Setup

The lecture uses a RAG chatbot (for a company, school or college) as its running example. Its flow is the familiar one: a **query** reaches the **retriever**, which is connected to a **vector database** and fetches relevant documents. The query and the retrieved documents are then handed to the **generator**, an LLM that writes the answer from that context.

Where Can It Break?

Two failure points are visible directly in this architecture, as shown in Figure 4.1:

- **Retriever failure** — it fetches the wrong documents, so the generator answers from wrong context.
- **Generator failure** — the retriever works, but the generator ignores the context and hallucinates a wrong answer.

Both must work for the application to work, so each gets its own eval pipeline.

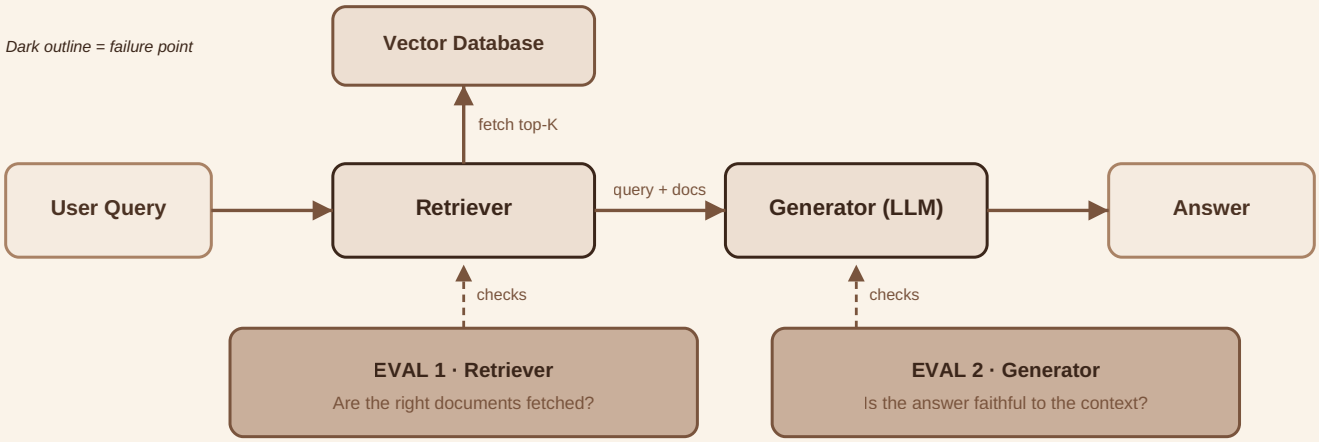


Figure 4.1 — A RAG application has two obvious failure points, each covered by its own component-level eval.

What Each Component Eval Checks

Component	Question the eval asks	Quality being checked
Retriever	Given a query, did we get the right, relevant documents?	Context relevance / retrieval quality
Generator	Given a context, is the answer generated from it?	**Faithfulness**, also called **groundedness**

FAITHFULNESS / GROUNDEDNESS: EXAMPLE

Suppose the question is “What is the duration of the machine learning course?” and the retrieved document says *3 weeks*. A faithful answer states that the course lasts three weeks. It must not add facts of its own, such as “it is a great course” or “you can also buy the Python course, which lasts four weeks.” The answer has to stay grounded in the context, so this eval checks that *context and answer are faithful to each other*.

> **INTERVIEW TIP**
>
> If asked how you would evaluate a RAG system, start by splitting it into retriever and generator, and name the quality each eval measures: relevance for the retriever, faithfulness (groundedness) for the generator. Then say that this is only the first layer.

> ↻ **Quick Revision — RAG Failure Points**
>
> • RAG flow: query → retriever (vector DB) → documents + query → generator (LLM) → answer.
>
> • The retriever and the generator are two separate failure points, so each gets its own eval.
>
> • The retriever eval asks whether the right documents were fetched for a query.
>
> • The generator eval checks faithfulness (groundedness): the answer must come only from the context.

3. When Every Component Passes but the System Fails

The Question

Assume both component evals report that the retriever and the generator work correctly. Does that guarantee the RAG application works? The lecture's answer is no, and it demonstrates why with a scenario.

Walkthrough: the "ML Course Duration" Trap

1. A user asks: *"What is the duration of the machine learning course?"*
2. The retriever is configured with **K = 5**, meaning it fetches the five most relevant documents from the vector database.
3. Documents D1–D4 are unrelated, and one of them states that the Python course lasts six weeks. D5 contains the correct fact: *the ML course is eight weeks*.
4. Did the retriever do its job? **Yes.** The correct document is inside the top K, so its component eval passes. (Reranking is deliberately set aside at this point.)
5. The query and all five documents go to the generator, whose system prompt tells it to give priority to the higher-ranked documents.
6. The generator picks up the "six weeks" fact from the top-ranked context and answers that the ML course lasts six weeks.

The answer is wrong, yet the generator did not hallucinate: the six-week fact really was in its context, and it followed its instructions. It was handed the wrong document at the top and mixed the facts up. So each component worked independently and **the pipeline still broke**.

The Fix: a Workflow-Level Eval

A third eval is needed, one that evaluates the **combination** of retriever and generator. It would flag this error and point to its cause: the most relevant document sits at the bottom of the priority order. The likely remedy is a **reranker**, which

reorders the results after retrieval so that D5 moves to the top and D1–D4 move down. The whole RAG pipeline then works. Figure 4.2 summarizes the case.

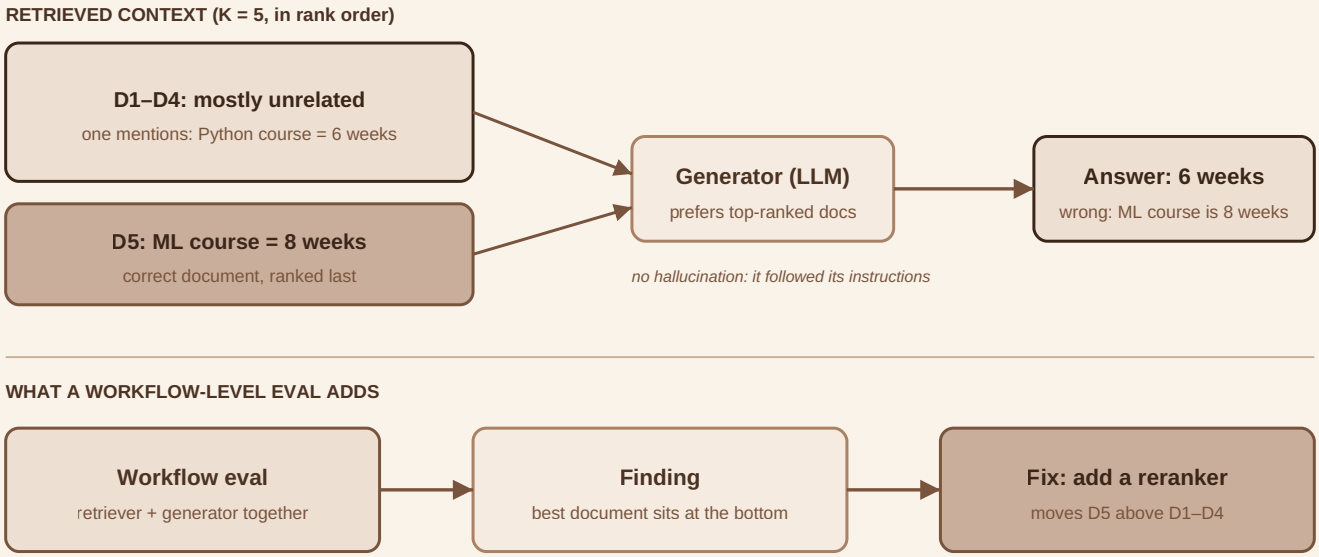


Figure 4.2 — Both components pass their own evals, but the correct document is ranked last, so the final answer is wrong. A workflow-level eval exposes this.

COMMON MISTAKE

Assuming that if every component passes its eval, the system is correct. Component evals test parts in isolation; they cannot see problems that only appear when the parts interact, such as ranking order feeding into generation.

↻ Quick Revision — Component vs Workflow Failures

- K is the number of documents the retriever fetches; the retriever passes if the correct one is within the top K.
- A generator told to prioritize higher-ranked documents can give a wrong answer without hallucinating.
- Component evals can all pass while the combined pipeline still returns a wrong answer.
- A workflow-level eval tests the retriever and generator together and can reveal ranking problems.
- A reranker reorders retrieved documents so the best one reaches the generator first.

4. Three Levels of Evals

Is Workflow-Level Enough?

Now suppose the retriever eval, the generator eval and the workflow eval all pass. The lecture asks whether the application is guaranteed to be good, and again the answer is no. Imagine the pipeline is correct but takes **10 seconds** to answer one question. A user would type a question and wait ten seconds, so the application is not fit for production even though every previous eval passed. A further eval is needed at the level of the **entire application**, one that keeps latency below a threshold.

The Three Levels

Failure points therefore exist at three levels, shown in Figure 4.3.

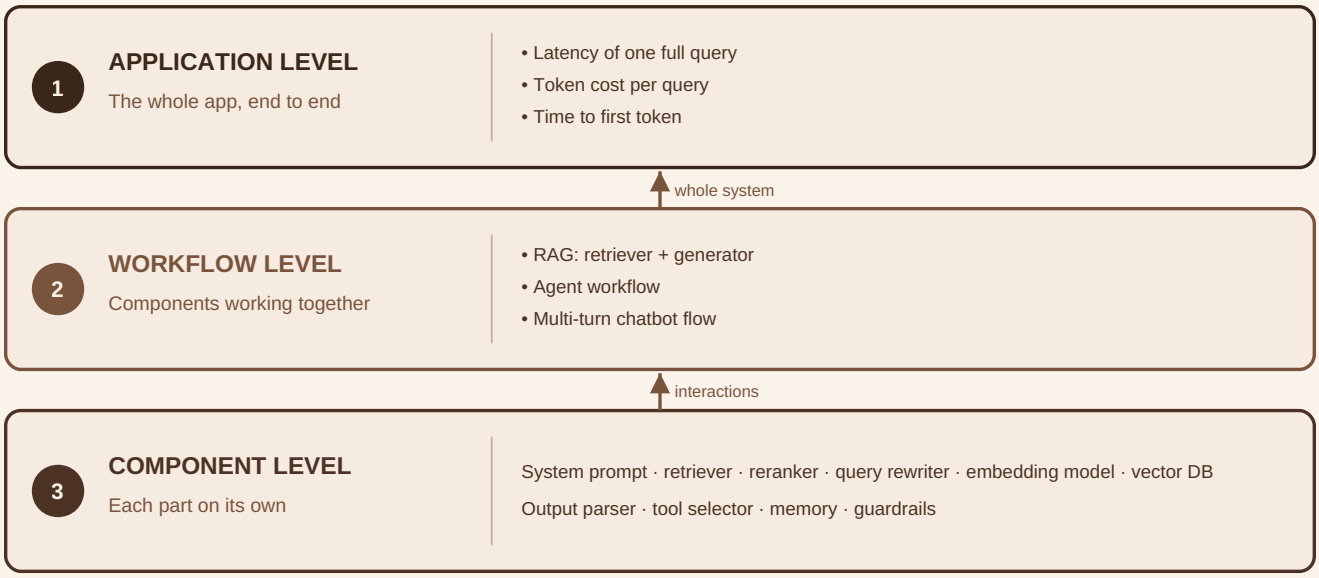


Figure 4.3 — Failure points exist at component, workflow and application level; each level needs its own evals.

Level	What can go wrong	Examples from the lecture
Component	Any individual part of the application can fail.	System prompt; in RAG: retriever, reranker, query rewriter, embedding model, vector DB; in structured-output apps: output parser; in agents: tool selector, memory, guardrails.
Workflow	Components work alone but fail in combination.	The RAG retriever + generator flow; an agent's workflow; a multi-turn chatbot's workflow.
Application	The whole app fails on a system-wide property.	End-to-end latency; token cost spent on one query; time taken to print the first token.

BEST PRACTICE

Place evals at all three levels. Each component gets its own eval pipeline, each multi-component workflow gets a combined eval, and the whole application gets evals for system-wide properties such as latency and token cost.

↻ Quick Revision — Three Levels

- Even with component and workflow evals passing, a 10-second response makes an app unfit for production.
- Component level: any single part (prompt, retriever, reranker, parser, tool selector, memory, guardrails) can fail.
- Workflow level: the interaction of components can fail even when each part is fine (RAG, agents, multi-turn chatbots).
- Application level: whole-app properties such as latency, cost per query and time to first token need their own evals.

5. Risk Categories: the Second Reason

Variations Inside Every Level

Sections 2–4 explained why evals are needed at several levels. There is one more reason: **within each of those levels, there are variations in what "good" means.** Each failure point has several aspects to it, and the lecture calls these **risk categories**. Three examples show how one component or workflow can be judged on different dimensions:

- **Application level, RAG chatbot:** the answer must be correct and helpful, but also *safe*. A chatbot must never reveal another user's phone number or email address.
- **Workflow level, retriever + generator:** the answer must be faithful and grounded, but producing it must not cost more than a threshold.
- **Component level, retriever:** it must fetch relevant documents, but fetching them must not take 5 or 10 seconds.

The Three Risk Categories

All risks are organized into three broad groups (Figure 4.4):

Category	What it tells you
Application Quality	Whether the app does its actual job well: it gives correct, relevant and complete answers to what the user asked.
Safety	Whether the output is free of harm: no toxic, dangerous or biased content, no private-data leak, and no way to jailbreak the app into doing what it should not.
Operations	Whether the app can run fast, cheaply and reliably once deployed.

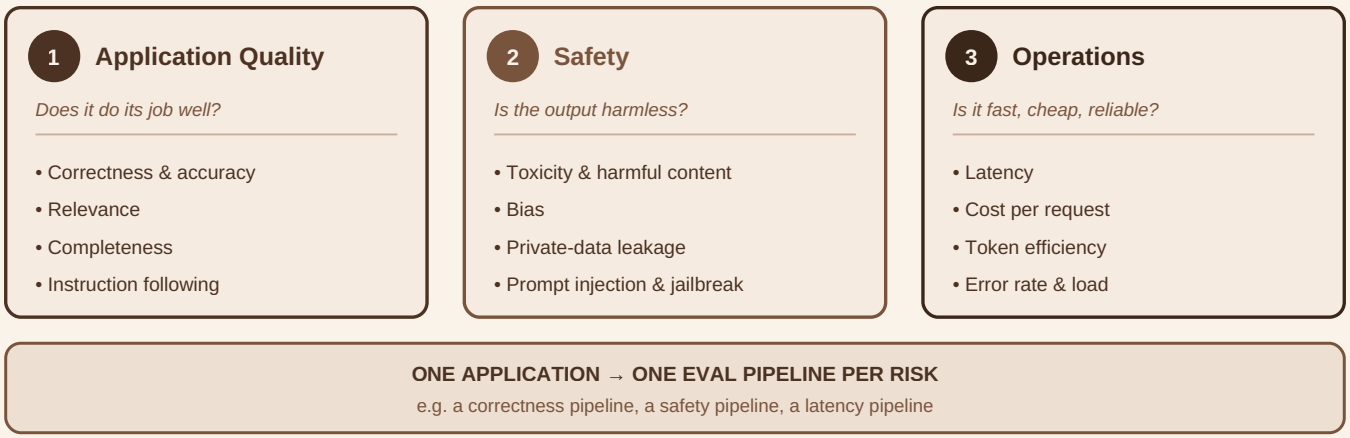


Figure 4.4 — Risks are organized into quality, safety and operations; each gets its own eval pipeline.

REAL WORLD INSIGHT

The same application can pass a correctness eval and still fail in production on safety or cost. This is why the lecture separates the pipelines: one for latency, one for safety, one for correctness, all on the same application.

↻ Quick Revision — Risk Categories

- Risk categories are the different aspects of failure attached to each failure point.
- Application Quality asks whether the app does its job well (correct, relevant, complete answers).
- Safety asks whether the output is harmless: no toxicity, bias, private-data leaks or jailbreaks.
- Operations asks whether the deployed app runs fast, cheap and reliably.

6. Risk Categories by Application Type

The lecture lists the risks that show up most often. The list is not exhaustive; it covers the ones you will see repeatedly when building your own applications.

Application Quality Risks

GENERAL LLM APP (E.G. A TEXT SUMMARIZER)

Risk category	What it asks
Correctness & accuracy	Is the generated summary accurate?
Relevance	Is the answer related to what was asked?
Completeness	Were all the questions answered?
Instruction following	Does the output respect the requested format and length?

RAG APPLICATION

Risk category	What it asks
Context relevance	Are the retrieved documents relevant? (the retriever’s job)
Retriever recall	Closely related to context relevance: does retrieval surface what is needed?
Groundedness & faithfulness	Is the answer generated from the given context, with nothing extra?
Citation accuracy	Can a generated line be traced to the document it was extracted from?

AGENT

Risk category	What it asks
Tool selection	Does the agent pick the right tool for the job?
Parameter correctness	Are the right parameters passed when a tool is called?
Task completion	Does the agent finish the task, or is the failure rate high?
Error recovery	If the agent goes wrong midway, can it recover?

MULTI-TURN CHATBOT

Risk category	What it asks
Context retention	How much of the earlier conversation does it remember?
Clarification behavior	When the user's input is ambiguous, does it ask for clarification?

Safety Risks

Dimension	What is checked
Toxicity	Whether the generated answer is toxic.
Harmful content	Content that should not come out, such as self-harm, weapons or illegal-activity content.
Bias	Whether the chatbot answers everyone the same way, or differently depending on the user's profile.
Private data leakage	Whether personal information such as credit-card details or contact details is revealed.
Prompt injection & jailbreak resistance	Whether someone can use prompts to make the app do something it should not.

Operational Risks

Dimension	What is checked
Latency	How long a response takes (also latency under load).
Cost per request	What each request costs to serve.
Token efficiency	How economically tokens are used.
Error / failure rate	How often requests fail.

INTERVIEW TIP

When asked how you would evaluate an agent or a chatbot, tailor the quality metrics to the application type: tool selection and error recovery for agents, context retention and clarification for multi-turn chatbots, then add the shared safety and operations checks.

↻ Quick Revision — Risks by Application Type

- General apps: correctness, relevance, completeness, instruction following.
- RAG: context relevance, retriever recall, groundedness/faithfulness, citation accuracy.
- Agents: tool selection, parameter correctness, task completion, error recovery.
- Multi-turn chatbots: context retention and clarification behavior.
- Safety and operations lists apply across application types.

7. From Risk Categories to Multiple Eval Pipelines

The risk categories translate directly into separate eval pipelines. It is the same application, but one pipeline measures latency, another measures safety and another measures correctness. Combined with the three levels of Section 4, this is why a real application accumulates many evals.

Reason	Source of multiplicity	Example from the lecture
Multiple failure points	Components, workflows, whole application	Retriever eval, generator eval, retriever + generator workflow eval, application latency eval.
Multiple risk categories	Quality, safety, operations	Correctness eval, safety eval (e.g. private-data leak), latency and cost evals.

The instructor’s conclusion is that whenever you build an LLM application, you will almost always (his words: 99.99% of the time) attach more than one evaluation pipeline to it. That is exactly the pattern used for the rest of this course: separate pipelines for separate questions. Two independent questions decide which evals you need: *where* can this application fail (component, workflow, application), and *what kind* of failure matters there (quality, safety, operations).

↻ Quick Revision — Multiple Eval Pipelines

- Risk categories map to separate eval pipelines on the same application.
- Failure points (where) and risk categories (what kind) together determine the set of evals.
- Almost every real LLM application needs more than one eval pipeline.

K. Key Takeaways

- **Several evals per application.** One LLM application normally carries multiple eval pipelines, for two independent reasons.
- **Multiple failure points.** A RAG app has at least two (retriever and generator), and each needs its own eval: relevance for the retriever, faithfulness (groundedness) for the generator.
- **Component evals are not enough.** With K = 5, a correct document ranked last can make a faithful generator answer wrongly; only a workflow-level eval exposes it, and a reranker is the likely fix.
- **Application-level evals.** A correct pipeline that takes 10 seconds is not deployable, so latency, token cost and time to first token need whole-application evals.
- **Three levels.** Component, workflow and application levels each contain their own failure points.
- **Multiple risk categories.** Every level can fail on quality, safety or operations, so these are evaluated by separate pipelines.
- **Risks depend on application type.** Summarizers, RAG apps, agents and multi-turn chatbots each add their own quality risks to the shared safety and operations lists.

R. Revision Sheet

Why multiple evals

Because an application has multiple failure points and each failure point has multiple risk categories.

Retriever eval

Given a query, checks whether the retriever fetched the right, relevant documents (context relevance).

Generator eval

Given a context, checks faithfulness / groundedness: the answer comes only from the context, with no added facts.

Top-K (K)

The number of most relevant documents the retriever fetches from the vector database.

Workflow-level eval

Evaluates how components work together, e.g. retriever + generator; catches errors such as a correct document ranked last.

Reranker

Reorders retrieved results after retrieval so the most relevant document is ranked highest.

Application-level eval

Checks whole-app properties such as end-to-end latency, token cost per query and time to first token.

Component level

Each part on its own: system prompt, retriever, reranker, query rewriter, embedding model, vector DB, output parser, tool selector, memory, guardrails.

Application Quality

Does the app do its job well: correct, relevant, complete answers.

Safety

Is the output harmless: toxicity, harmful content, bias, private-data leakage, prompt injection and jailbreak resistance.

Operations

Can it run fast, cheap and reliably: latency, cost per request, token efficiency, error rate, latency under load.

I. Interview Questions

Q1. Why does a single LLM application typically need more than one evaluation pipeline? Give the two reasons.

Q2. In a RAG chatbot, what are the two obvious failure points, and what does each component-level eval check?

Q3. Explain faithfulness (groundedness) with an example in which the generator adds facts that are not in the retrieved context.

Q4. With K = 5, the retriever fetches the correct document at rank 5 and the generator returns a wrong answer. Did either component fail? Explain.

Q5. Why does a workflow-level eval catch problems that component-level evals miss, and what fix does the lecture suggest in this case?

Q6. Your RAG pipeline passes retriever, generator and workflow evals but takes 10 seconds per answer. What kind of eval is missing, and what would it measure?

Q7. Describe the component, workflow and application levels of evaluation for an agent.

Q8. What are the three risk categories, and what does each cover? Give one metric for each.

Q9. Which quality risk categories matter most for an agent, and which for a multi-turn chatbot?

Q10. A chatbot gives correct answers but sometimes reveals another user's contact details. Which risk category is this, and how would you set up evals so it is caught?

F. Further Reading

- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — arXiv:2005.11401 (https://arxiv.org/abs/2005.11401)
- Es et al., "RAGAS: Automated Evaluation of Retrieval Augmented Generation" — arXiv:2309.15217 (https://arxiv.org/abs/2309.15217)